

The 8085 Master Manual

Volume III: The Heartbeat
(Timing & Orchestration)

A Grassroots Engineering Guide

Structuring time, resolving bus contention, and synchronizing chaos.

"Logic determines what happens; Time determines when."

Chapter 1: The Chronology of Silicon

1.1 The Necessity of a Clock

In Volume I, we established the physical buses. In Volume II, we constructed the internal registers and the ALU. However, if all these components were allowed to operate simultaneously, the system would collapse into electrical chaos. Data from the Accumulator would collide with data from the Memory on the Internal Data Bus.

To prevent this, the microprocessor operates as a **Synchronous Sequential Machine**. Every internal latch, every buffer enable, and every memory read/write operation is strictly governed by a central metronome: **The System Clock**.

1.2 The Hierarchy of Time

Time in the 8085 architecture is not measured in seconds; it is measured in distinct hierarchical units determined by the processor's internal state machine.

1. THE T-STATE (TIME STATE)

A T-State is the most fundamental, indivisible unit of time in the microprocessor. It is exactly equivalent to one period of the internal system clock. If the 8085 is driven by a 2 MHz system clock, the duration of one T-State is exactly $T = \frac{1}{f} = \frac{1}{2 \times 10^6} = 0.5 \mu\text{s}$. Every single microscopic electrical event occurs at the edge of a T-State.

2. THE MACHINE CYCLE

A Machine Cycle is the time required by the microprocessor to complete one specific, singular interaction with the external world (such as reading an opcode, writing a byte to memory, or reading an I/O port). A Machine Cycle is composed of

3 TO 6 T-STATES

.

3. THE INSTRUCTION CYCLE

An Instruction Cycle is the total time required to fetch, decode, and completely execute a single assembly language instruction (e.g., **STA 8050H**). An Instruction Cycle consists of

1 TO 5 MACHINE CYCLES

, depending on the complexity of the instruction.

1.3 Clock Generation

The 8085 contains an internal clock generator circuit. It requires an external crystal oscillator connected across the X_1 and X_2 pins. To ensure a perfect 50% duty cycle for its internal flip-flops, the 8085 **divides the input crystal frequency by 2**.

If you connect a standard 4 MHz crystal to the 8085, the internal system clock operates at 2 MHz. The processor also provides a **CLK OUT** pin, outputting this 2 MHz signal to synchronize peripheral devices on the motherboard.

Chapter 2: The Opcode Fetch Cycle

Every instruction executed by the 8085 begins identically: the processor must read the instruction code (the Opcode) from memory and decipher what it means. This fundamental operation is the **Opcode Fetch Machine Cycle** (usually denoted as **M₁**).

2.1 The 4-T State Choreography

Unlike a simple memory read, an Opcode Fetch generally requires **4 T-states**. Let us break down the exact sequence of electrical events mandated by the Timing and Control Unit.

T1: The Address Phase

At the start of **T₁**, the processor dumps the contents of the Program Counter (PC) onto the address bus. The high-order byte goes to **A₈-A₁₅**, and the low-order byte goes to the multiplexed **AD₀-AD₇** pins. Crucially, the **ALE (Address Latch Enable)** signal is pulsed HIGH. The processor also sets its status pins to announce its intent: **IO/M' = 0**, **S₁ = 1**, **S₀ = 1**.

T2: The Wait and Read Phase

At the start of **T₂**, the **ALE** signal has fallen, meaning the external latch has locked the lower address. The 8085 floats its **AD₀-AD₇** pins (high-impedance) to prepare them for incoming data. It then drives the **RD'**

control signal LOW, commanding the addressed memory chip to place the opcode onto the data bus.

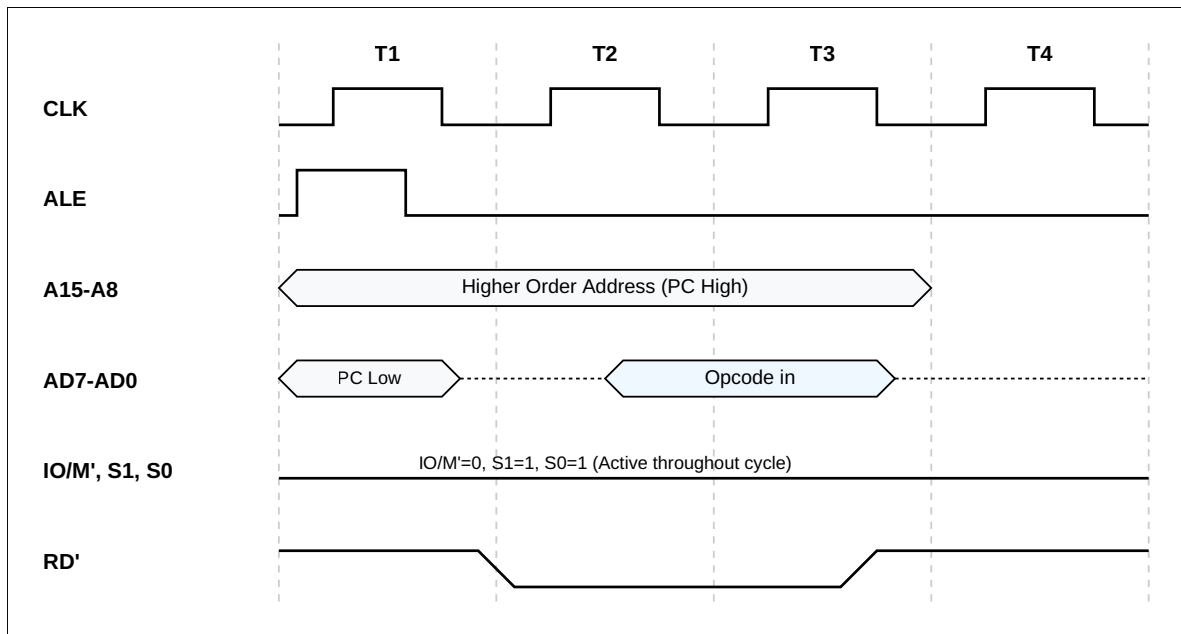
T3: The Acquisition Phase

The memory chip has placed the 8-bit opcode on the AD₀-AD₇ pins. During T₃, the CPU latches this binary data directly into its internal **Instruction Register (IR)**. Once safely inside, the CPU drives the **RD'** signal back HIGH, terminating the read operation.

T4: The Decode and Execution Phase

During T₄, the external buses are idle. The 8085 passes the opcode from the Instruction Register into the Instruction Decoder. The decoder determines the nature of the instruction and initializes the Timing and Control Unit to generate the subsequent machine cycles (if the instruction requires them).

2.2 Visualizing the Opcode Fetch



*Note: Certain exceptionally complex instructions (e.g., **CALL** or **INX H**) require additional internal decoding time, extending the Opcode Fetch cycle to **6 T-states**.*

Chapter 3: Memory and I/O Cycles

Once the instruction is decoded, the CPU may discover it needs to transfer additional data. For example, the instruction `STA 8050H` requires the CPU to read the two address bytes (80H and 50H) from memory, and then write the Accumulator's contents to that newly discovered address. These operations are handled by **Memory Read** and **Memory Write** machine cycles.

3.1 The 3-T State cycles

Unlike the Opcode Fetch, pure data transfer cycles do not require internal decoding time in the Instruction Register. Therefore, Memory Read, Memory Write, I/O Read, and I/O Write cycles are all completed in exactly **3 T-states**.

Memory Read Machine Cycle

Status: $IO/M' = 0$, $S_1 = 1$, $S_0 = 0$.

T1: Address is placed on the bus; ALE pulses high to latch the lower address.

T2: The RD' signal goes LOW. The memory chip decodes the address and retrieves the data.

T3: The memory chip places data on the AD_0 - AD_7 bus. The CPU latches the data into an internal register (like the Accumulator or a general-purpose register). RD' goes HIGH.

Memory Write Machine Cycle

Status: $IO/M' = 0$, $S_1 = 0$, $S_0 = 1$.

T1: Target address is placed on the bus; ALE pulses high.

T2: The CPU places the data (e.g., from the Accumulator) onto the AD₀-AD₇ bus. The WR' signal goes LOW.

T3: The memory chip absorbs the data currently sitting on the data bus. WR' goes HIGH, signaling the end of the write operation.

I/O CYCLES VS. MEMORY CYCLES

An I/O Read or I/O Write machine cycle is timing-identical to a Memory cycle (3 T-states, identical ALE and RD'/WR' sequencing). The *only* difference is the state of the

IO/M'

pin.

During an I/O cycle, $\text{IO/M}' = 1$, which physically disables all memory chips on the motherboard and enables the peripheral decoders. Furthermore, because I/O ports only use 8-bit addresses, the 8-bit port address is duplicated on both the AD₀-AD₇ and A₈-A_{15} buses.

Chapter 4: The Problem of Slow Memory (Wait States)

The timing diagrams presented in Chapter 3 assume a perfect world where external devices react instantly. In reality, memory chips have a physical limitation known as **Access Time** (t_{acc}).

The Synchronization Crisis: The 8085 processor drives the RD' signal LOW in T_2 and locks the data from the bus into its internal registers at the end of T_3 . At 2 MHz, the time between these two events is roughly 500 nanoseconds. If a cheap EPROM chip takes 800 ns to retrieve its data, the CPU will latch invalid "garbage" data off the bus before the EPROM is ready.

4.1 The READY Signal and T_{WAIT}

To prevent this, the 8085 architecture includes the **READY** input pin, allowing for asynchronous hardware synchronization.

During the T_2 state of every machine cycle, the 8085 samples the voltage level on the **READY** pin.

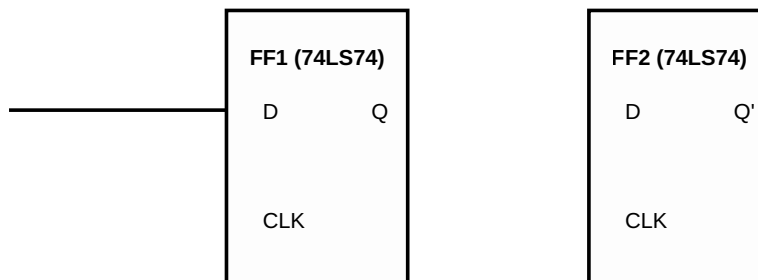
- If $\text{READY} = 1$ (HIGH), the CPU proceeds normally to T_3 .
- If $\text{READY} = 0$ (LOW), the CPU halts its internal state machine clock. Instead of entering T_3 , it inserts a **Wait State** (T_{WAIT}).

During $T_{\{\text{WAIT}\}}$, the CPU "freezes". It holds the address on the address bus perfectly stable. It holds the RD' or WR' signal perfectly stable. It does absolutely nothing but check the **READY** pin again at the next clock edge. It will insert infinite Wait States until the slow memory chip finally drives the **READY** pin HIGH, signaling it has placed valid data on the bus.

4.2 Hardware Generation of Wait States

How do we ensure the **READY** pin goes LOW at exactly the right time? We build a hardware **Wait State Generator** using D-Flip-Flops (like the 74LS74).

The objective is to pull **READY** low during T_2 and return it high before T_3 to insert exactly one Wait State (delaying the CPU by one clock cycle). We use the **ALE** pulse from T_1 as our trigger.



Circuit Logic: ALE goes high in T_1 . The clock edge transitions into T_2 , latching the '1' into FF1. On the next clock edge, the '1' moves to FF2. Since we take the inverted output Q' of FF2, **READY** is pulled LOW during T_2 . By the

next clock, ALE is already 0, pushing a '0' through the chain and returning READY to HIGH, thereby inserting exactly one T_{WAIT} .

Chapter 5: Synthesis - Tracing the Execution

To master the concept of the Machine Cycle, we must trace a complex instruction from fetch to completion, counting every single clock pulse.

EXECUTION TRACE: STA 8050H (STORE ACCUMULATOR DIRECT)

The **STA** instruction stores the 8-bit contents of the Accumulator into a directly specified 16-bit memory location. It is a 3-byte instruction: Byte 1 is the Opcode (**32H**), Byte 2 is the lower address (**50H**), and Byte 3 is the upper address (**80H**).

TOTAL T-STATES: 13 | TOTAL MACHINE CYCLES: 4

- **M_1: OPCODE FETCH (4 T-STATES)**

The CPU places the PC on the address bus. RD' goes low. Memory returns the opcode **32H**. The CPU decodes it and realizes it needs two more bytes to form a complete address.

- **M_2: MEMORY READ (3 T-STATES)**

The CPU increments the PC, places it on the address bus. RD' goes low. Memory returns the byte **50H**. The CPU stores this in the internal Z register (Temp).

- **M_3: MEMORY READ (3 T-STATES)**

The CPU increments the PC again. RD' goes low. Memory returns the byte **80H**. The CPU stores this in the internal W register (Temp). The WZ register pair now holds the full target address: **8050H**.

- **M_4: MEMORY WRITE (3 T-STATES)**

The CPU places the contents of the WZ register pair (**8050H**) onto the address bus. It dumps the contents of the Accumulator onto the data bus. It drives the WR' signal LOW, commanding the memory to absorb the data.

At 2 MHz, this entire symphony of 13 clock pulses executes in exactly 6.5 microseconds.

The 8085 executes millions of these cycles every second. By strictly structuring the sequence of events using ALE to multiplex buses, and T-States to govern control signals, the architects of the 8085 managed to extract immense logical complexity from extremely constrained hardware limitations.

With the physical interface built, the internal anatomy constructed, and the heartbeat synchronized, we have a fully functioning silicon entity. The next phase is to connect it to an ecosystem of external peripherals—the art of **Interfacing**.

End of Volume III